

Character Detection, Tracking, and Identity Matching in Super Smash Bros. Melee

Brody Clark

August 29, 2026

1 Introduction

Unlike standard object-tracking tasks that rely on stationary camera inputs, gameplay footage introduces additional complexity, including camera panning, parallax backgrounds, and sudden changes in character animation and color profiles. While some modern games expose telemetry through APIs or replay files, many still provide no standardized interface for extracting gameplay information. Consequently, all game state information must be inferred directly from rendered video.

Detecting and tracking characters in Super Smash Bros. Melee presents several challenges that differ significantly from conventional object tracking problems. Unlike surveillance or autonomous driving datasets, the in-game camera continuously pans and zooms throughout each match. Characters frequently overlap, become temporarily occluded, leave the visible frame entirely, and rapidly change animation states. Furthermore, multiple characters may share similar colors or occupy nearby locations, making appearance-based identification alone unreliable.

This project investigates the use of classical computer vision techniques for detecting, tracking, and identifying player characters in Super Smash Bros. Melee. The system combines image segmentation, contour analysis, temporal tracking, and different combinations of feature descriptors to explore how effectively lightweight, classical methods can analyze gameplay footage.

2 Background

Multiple object tracking has traditionally included a combination of object detection and temporal data association. One influential lightweight tracking framework is Simple Online and Realtime Tracking (SORT), introduced by Bewley et al. [1], which combines Kalman filtering with the Hungarian assignment algorithm to maintain object identities across consecutive frames. SORT demonstrated that effective tracking can be achieved without computationally expensive learning models when reliable detections are available.

Although object tracking has been extensively studied for surveillance and autonomous navigation, comparatively little work focuses on extracting gameplay information directly from rendered video. Kozłowski et al. [2] demonstrated that classical computer vision methods can be effective for real-time analysis of game footage. Their results showed that Local Binary Pattern (LBP) descriptors can provide effective recognition of visual game elements while maintaining low computational costs, and that classification performance can be further improved by incorporating temporal consistency measures.

Shape-based feature extraction provides another lightweight feature for object classification. Amraji et al. [3] proposed representing object silhouettes using normalized Fourier Descriptors, which produce feature vectors that are invariant to translation, rotation, and scaling. Zhang and Lu [4] further evaluated several contour signatures for Fourier Descriptor generation, and found that centroid-distance signatures provided strongest retrieval performance among those evaluated. Hu et al. [5] derived a set of seven non-linear combinations of normalized central moments that exhibit invariance under translation, scale, and rotation. These techniques provide a robust mechanism for comparing character silhouettes despite animation changes and viewpoint variation.

Unlike many traditional vision datasets, Super Smash Bros. Melee introduces additional challenges. Dynamic camera movement breaks assumptions of stationary backgrounds, while rapid character movement, overlapping actors, particle effects, and stage-specific visual changes complicate segmentation and tracking. Maintaining persistent character identities therefore requires integrating temporal prediction with appearance-based matching to recover identities after occlusions or off-screen movement.

3 Approach

The proposed system is organized as a modular processing pipeline consisting of three independent components: a detector, a tracker, and a matcher. Each module performs its specific task before forwarding its results to the next stage. This allows individual components to be improved without modifying the remainder of the system. The pipeline ultimately constructs a unified game state containing character screen-space locations, identities, and current animations.

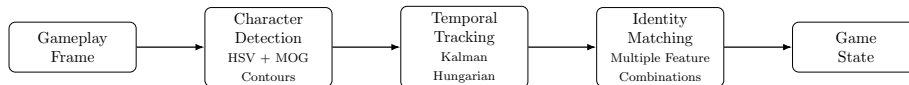


Figure 1: Modular architecture of the proposed gameplay analysis system. The detector, tracker, and identity matcher operate as separate stages connected by intermediate representations.

3.1 Character Detection

The first stage identifies candidate character locations from raw gameplay frames using classical computer vision techniques implemented in OpenCV. Segmentation is performed through carefully constructed HSV color masks tailored to each included stage. Combining narrow color ranges suppresses large portions of the background without masking the dominant character colors.

Because color analysis alone cannot distinguish characters from the environment and overlays, foreground extraction is supplemented using a Mixture of Gaussian (MOG) motion mask. The combination of HSV segmentation and motion detection significantly reduces false positives by emphasizing moving foreground regions while retaining stationary character pixels that remain visually distinctive.

Contours extracted from the resulting binary masks are converted into candidate bounding boxes. Geometric filtering based on contour area and shape removes environmental objects, particle effects, and residual background artifacts before detections are passed to the tracking stage.

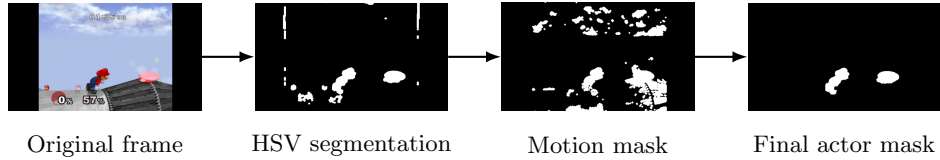


Figure 2: Character detection pipeline from the original gameplay frame to the final actor mask. Color-based segmentation and motion information are combined to suppress background regions and isolate candidate character pixels.

3.2 Tracking

Once character detections have been generated, temporal consistency is maintained using a multiple-object tracking framework inspired by the SORT architecture. Each active target maintains an independent Kalman filter that predicts its expected position in the following frame using previous observations.

Predicted object locations are associated with newly detected bounding boxes using the Hungarian assignment algorithm. This optimization minimizes overall assignment cost and preserves existing trajectories where possible. Combining Kalman prediction with optimal frame-to-frame assignment enables the tracker to maintain continuous tracklets despite short-term occlusions or temporary detection failures.

3.3 Feature Extraction and Identity Matching

While the tracker preserves local trajectories, determining character identity requires comparing visual appearance with established templates. To investigate

which visual characteristics provide the most reliable identity information, several feature representations were evaluated, including histograms in HSV color space, normalized Fourier Descriptors, Local Binary Patterns, and Hu Moments. These features capture complementary aspects of character appearance, including color, shape, and local texture.

Shape information is represented using normalized Fourier Descriptors computed from character contours. Fourier Descriptors provide compact representations that are largely invariant to translation, rotation, and scale, making them suitable for comparing character silhouettes across varying camera positions and animation frames. Hu Moments were also evaluated as a complementary shape descriptor, and provide additional invariance to translation, rotation, and scale while representing the overall geometric structure of the detected region. Their use follows established approaches to invariant shape representation.

Texture information is represented using Local Binary Pattern (LBP) descriptors, which capture local texture structure. Color information is represented using histograms computed in HSV color space. Although palette changes can alter a character’s color distribution, the experiments investigated whether retaining color information could provide useful discriminative information for character identification. Feature combinations were evaluated to determine the tradeoff between color invariance and discriminative appearance information.

Each feature representation is compared against corresponding character templates to produce an appearance-based similarity score. Different combinations of features and template palettes were evaluated to determine which representation provided the most reliable character classification. The resulting identity predictions are combined with the tracker output to assign persistent global identities. By separating short-term tracking from appearance-based identity matching, the system can recover character identities following extended occlusions or periods in which a character leaves the visible frame.

4 Results

Footage was captured from 10 seconds of gameplay involving two characters, Mario and Kirby, across three stages: Corneria, Temple, and Venom. The recording in Venom introduces a palette swap for Mario to evaluate the robustness of character classification to changes in appearance. All recordings were captured at a fixed resolution of 1280×720 pixels and a frame rate of 30 frames per second. The predicted game state and per-frame processing time were recorded and used to derive the evaluation metrics described below.

The character matching database was constructed from manually extracted animation frames. Because manually extracting the complete set of animation frames was impractical, the database contains a core subset of the available character animations rather than an exhaustive collection. The evaluation considered 18 animation categories: idle, walk, run, jump, attack, side attack, up attack, special, side special, up special, down special, damaged, guard, crouch,

down attack, fall, recover, and dodge. Additionally, all templates include only the default color palette for each evaluated character. These limitations are important to consider when interpreting animation-matching performance, since gameplay frames may contain poses, visual effects, or colors that are not represented in the template database.

The evaluation was divided into detection, tracking, character classification, animation matching, and runtime performance. Detection was evaluated using bounding-box overlap with manually annotated ground truth data, while tracking was evaluated using Multi-Object Tracking Accuracy (MOTA). Character and animation recognition were evaluated against the corresponding manually assigned labels. Finally, runtime performance is measured in frames per second (FPS).

4.1 Detection

Detection performance was evaluated using precision, recall, and F1-score based on an Intersection over Union (IoU) threshold of 0.5. The results are summarized in Table 1.

Table 1: Detection performance metrics.

Stage	Frames	Visible GT Objects	TP	FP	FN	Precision	Recall	F1
Corneria	303	606	417	182	189	0.6962	0.6881	0.6921
Temple	312	606	373	176	233	0.6794	0.6155	0.6459
Venom	315	628	401	331	227	0.5478	0.6385	0.5897

Corneria produced the strongest overall detection performance, with an F1-score of 0.6921. The detector achieved a recall of 0.6881 while precision was 0.6962. Temple produced similar results, with an F1-score of 0.6459.

Venom produced substantially more false positives than the other two stages. Although its recall of 0.6385 was an improvement from Temple, precision decreased to 0.5478 because the detector generated 339 false-positive regions. This resulted in the lowest F1-score of 0.5897.

The difference between stages suggests that the effectiveness of the segmentation approach is strongly dependent on the visual characteristics of the stage. The detector relies heavily on combinations of narrow HSV ranges designed to suppress stage colors while preserving character pixels. Corneria and Temple provided conditions under which these masks were relatively effective, whereas Venom produced more regions that satisfied the detector’s visual and geometric criteria despite not containing characters. This result is consistent with the observation during development that segmentation quality was one of the most important factors affecting the remainder of the pipeline.

4.2 Tracking

Tracking continuity was evaluated using Multi-Object Tracking Accuracy (MOTA). The MOTA score is computed as:

$$MOTA = 1 - \frac{\sum FN + FP + IDSW}{\sum g}$$

Where:

1. FN is the false negatives.
2. FP is the false positives.
3. $IDSW$ is the identity switches.
4. g is the ground truth number of real characters on screen in the given frame.

As shown in Table 2, the tracking system maintained relatively consistent identities across all three stages, although identity switches occurred throughout each sequence. Corneria produced 10 identity switches, while Temple and Venom each produced 11. The relatively small difference in identity switches between stages suggests that the Kalman filter and Hungarian assignment approach were generally effective at maintaining temporal identities when reliable detections were available.

Table 2: Tracking performance metrics.

Stage	ID switches	MOTA
Corneria	10	0.3713
Temple	11	0.3069
Venom	11	0.0939

The relatively low MOTA values, however, demonstrate that maintaining identity is not equivalent to achieving high overall tracking accuracy. MOTA incorporates false positives and false negatives in addition to identity switches. Consequently, the detection failures observed in the previous section directly reduce the tracking score even when the tracker itself does not frequently swap identities.

Corneria produced the highest MOTA at 0.3713, followed closely by Temple at 0.3069. Venom produced a substantially lower MOTA of 0.0939. This corresponds with its substantially higher false-positive count and lower detection precision. The results therefore suggest that the largest limitation of the tracking pipeline is not frequent identity swapping, but rather the quality and consistency of the detections supplied to the tracker.

Character overlap and rapid changes in motion, particularly during attack animations, likely contributed to identity switches. When characters overlap

or change direction abruptly, the predicted position from the Kalman filter can become less representative of the character’s subsequent location, increasing the likelihood of an incorrect frame-to-frame association. Dynamic visual effects may further complicate this association by introducing additional regions that resemble character detections.

Overall, the tracking results suggest that the temporal association approach was reasonably effective once reliable character detections were established. The larger variation in MOTA between stages highlights the dependence of tracking performance on the quality and consistency of the upstream detection stage.

4.3 Character Classification

To evaluate the impact of different visual descriptors, character classification was benchmarked across various feature combinations on true-positive detections produced by the detector. Classification was considered correct when the system assigned the correct character identity between Mario and Kirby.

Comparisons were made between shape-based features including Fourier Descriptors (FD) and Hu Moments (Hu), texture-based features including Local Binary Patterns (LBP), and color-based features including full HSV histograms (HSV) an saturation-only histograms (SAT). Table 3 details the classification accuracy across all three test stages.

Table 3: Character classification accuracy across feature combinations.

Configuration	Features	Stage Accuracy (%)			
		Cor.	Tem.	Ven.	Average (%)
Color-Only	HSV	83.69	98.66	48.38	76.91
Shape-Only	FD, Hu	74.34	64.08	50.87	63.10
Texture + Shape	FD, Hu, LBP	73.62	62.20	56.36	64.06
Color-Invariant	FD, LBP, SAT	84.41	95.44	49.38	76.41
Texture + Shape + Color	FD, LBP, HSV	86.57	96.87	53.12	78.85
Shape + Color 1	FD, HSV	85.37	96.25	61.10	80.91
Shape + Color 2	Hu, HSV	84.41	97.05	61.85	81.10
Full Ensemble	FD, Hu, LBP, HSV	84.89	94.64	58.35	79.29

The results in Table 3 show that the combinations that included only shape (FD, Hu) and color (HSV) were most effective at distinguishing the two characters on all stages. Corneria and Temple had the highest accuracies across all tests which is likely due to the template dataset relying on the default color palette for both Kirby and Mario, whereas Venom included a color palette swap for Mario. The best combination overall is Hu moments and HSV histograms with an average character classification accuracy across all stages of 81.10% while Fourier Descriptors and HSV histograms had a similar result of 80.91%. The worst combinations were the Shape-Only and Texture + Shape combinations with accuracies of 63.10% and 64.06% respectively.

These results indicate color-based features contain important information that is not captured by combining texture (LBP) and saturation, which goes against original expectations that character classification across color palette variations would perform better with color-invariant features.

The substantially lower accuracy on Venom indicates that the character matching stage was more sensitive to segmentation quality than the results on the first two stages suggest. Incorrect or contaminated regions can alter both contour-based and texture-based descriptors, which make a correct character difficult to distinguish from the other. Additionally, stages which change dramatically over time, such as Venom, introduce a unique challenge in formulating effective HSV masks that account for all aspects of the stage at all times. This also demonstrates the interaction between the detector and matcher where errors introduced during segmentation propagate into later stages rather than being independently corrected by the classifier.

4.4 Animation Matching

In addition to identifying the character, the system attempted to identify the corresponding animation state using the available pre-compiled sprite templates. Animation matching was considerably less reliable than character classification, as accuracy remained at or below 15% on all three stages. The results are shown in Table 4.

Table 4: Animation classification accuracy across feature combinations.

Configuration	Features	Stage Accuracy (%)			
		Cor.	Tem.	Ven.	Average (%)
Color-Only	HSV	14.15	10.19	3.74	9.36
Shape-Only	FD, Hu	3.36	7.24	6.48	5.69
Texture + Shape	FD, Hu, LBP	1.68	8.58	6.48	5.58
Color-Invariant	FD, LBP, SAT	6.71	11.26	6.73	8.23
Texture + Shape + Color	FD, LBP, HSV	6.00	12.87	5.74	8.29
Shape + Color 1	FD, HSV	7.43	12.33	4.49	8.08
Shape + Color 2	Hu, HSV	14.15	10.19	2.49	8.94
Full Ensemble	FD, Hu, LBP, HSV	6.47	9.92	5.74	7.38

This result is largely attributable to limitations in the template dataset and challenges associated with differentiating similar poses among different animations rather than simply the choice of matching descriptors. The animation database was constructed from pre-compiled animation frames, and only a subset of the complete animation space was practical to extract. The available templates therefore do not represent every possible animation frame, pose, or transition that can occur during gameplay.

4.5 Runtime

Runtime performance was measured by taking the average FPS of each frame’s processing with Python’s `time.perf_counter()`. Measurements were taken and compiled for three trials per stage and the mean $\pm$ std were calculated across 2 machine, and are shown in Table 5. On the weaker configuration (Ryzen 5 PRO 4650U, 2.1 GHz, 6 cores, 16 GB) the pipeline achieved a top frame rate of 85.94 ± 10.81 FPS when classification features were comprised on HSV histogram only, and a bottom frame rate of 55.21 ± 3.83 when the full ensemble of features was used. On the more capable configuration (Ryzen 5 9600X, 3.9 GHz, 6 cores, 32 GB) the pipeline achieved a top frame rate of 210.31 ± 28.66 FPS when classification features were comprised on HSV histogram only, and a bottom frame rate of 173.14 ± 19.12 when the full ensemble of features was used.

Both devices achieved FPS higher than the recording’s FPS of 30, meaning for gameplay recorded at console-native resolution and framerate, this pipeline is capable of achieving realtime performance. For gameplay captured at 60 FPS, the features used for classification should be taken into consideration if realtime performance is required.

Table 5: Average runtime (FPS) on two test machines.

Configuration	Features	Average Stage Performance (FPS)	
		Ryzen 5 PRO 4650U	Ryzen 5 9600X
Color-Only	HSV	85.19 ± 10.27	210.31 ± 28.66
Shape-Only	FD, Hu	85.94 ± 10.81	211.43 ± 26.59
Texture + Shape	FD, Hu, LBP	69.77 ± 8.55	181.26 ± 20.06
Color-Invariant	FD, LBP, SAT	66.12 ± 6.74	180.13 ± 19.17
Texture + Shape + Color	FD, LBP, HSV	65.17 ± 5.96	175.07 ± 18.47
Shape + Color 1	FD, HSV	76.49 ± 7.88	202.85 ± 25.10
Shape + Color 2	Hu, HSV	74.58 ± 10.29	205.97 ± 25.19
Full Ensemble	FD, Hu, LBP, HSV	55.21 ± 3.83	173.14 ± 19.12

5 Discussion

The evaluation shows that reliable segmentation is the dominant factor affecting end-to-end performance. Carefully combined, narrow HSV masks removed large portions of stage background while preserving character pixels and yielded the best detection results on Corneria and Temple. However, stages with strongly time-varying visual content, such as Venom, are difficult to characterize with a small set of narrow HSV ranges as either the ranges must be widened, which increases likelihood of removing character pixels, or greatly enumerated, which increases computation and can harm runtime performance. This tradeoff motivated the practical optimization of running the detector on a reduced-resolution

frame to lower computational cost while preserving enough detail for segmentation. This method of optimization outperformed tracker prediction-driven localized searching, as knowing when to perform a localized versus global frame search proved to be a significant challenge.

Many classification failures stem from dynamic visual effects and animation ambiguity. Several animations share near-identical poses on some frames such as running vs. certain attack frames as seen in Figure 3, so appearance-only matching will frequently confuse states unless temporal or player-state information is available to rule out infeasible transitions. Similarly, VFX, charging glows, and damage flashes create contaminated regions that corrupt contour- and texture-based descriptors. These explain why animation matching accuracy remained below 10% and why character classification dropped on Venom where false positives and contaminated masks were common.

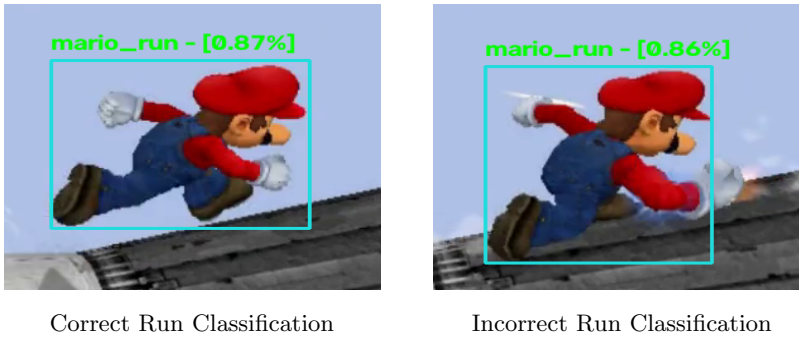


Figure 3: Results of mis-classifying Mario performing special-up attack as running due to having similar poses.

Practical improvements fall into two broad strategies. First, strengthen detection and temporal reasoning with a more complete template database and multi-frame temporal consistency checks. This could reduce contamination and stabilize descriptors. Second, relax the assumption of appearance-only matching with a player state model that could disambiguate visually similar frames by incorporating motion priors and feasible action sequences.

Performance-wise, the detector proved to be the main bottleneck. Operating on scaled-down frames provided large computational gains with acceptable detection degradation in many cases.

In summary, HSV masking was essential for producing usable detections but introduces a difficult tradeoff between specificity and computational cost on dynamic stages. Addressing animation ambiguity and dynamic VFX would require combining stronger temporal models with more complete templates, and potentially including player state models to predict feasible animation changes. While animation matching is insufficient for real gameplay analysis, this pipeline could prove useful in extraction of gameplay metadata such as active fighter count, damage counters, and remaining stock count.

6 Conclusion

This project explored the use of classical computer vision techniques for detecting, tracking, and identifying characters in Super Smash Bros. Melee gameplay footage. The implemented pipeline combined HSV-based segmentation, motion masking, contour analysis, Kalman filtering, Hungarian data association, Fourier Descriptors, Local Binary Patterns, and saturation histograms to investigate how effectively lightweight image processing techniques could be applied to a dynamic video game environment.

The results demonstrate that the pipeline was able to detect and maintain character identities with reasonable performance on Corneria and Temple. Corneria achieved the highest detection F1-score of 0.7018, while Temple achieved a comparable score of 0.6778. The tracking system also produced very few identity switches across all tested stages. These results indicate that the Kalman filter and Hungarian assignment approach were generally effective at maintaining temporal identity when reliable detections were available. However, the substantially lower MOTA on Venom demonstrates how detection errors can propagate through the remainder of the tracking pipeline.

One of the most important observations from the project was the importance of segmentation quality. Carefully combining multiple narrow HSV thresholds and combining results with a Mixture of Gaussian motion mask proved substantially more useful than relying on a single broad color metric. This allowed stage-specific colors to be removed while preserving character pixels and was particularly effective on mostly static stages like Corneria and Temple. In contrast, Venom’s changing visual content made it more difficult to construct a small set of HSV masks that remained effective throughout the sequence. The resulting false positives and contaminated character regions reduced both detection and classification performance.

The feature-matching experiments also demonstrated the usefulness and limitations of classical descriptors. The combination of HSV histograms and shape descriptors including Fourier Descriptors and Hu Moments provided reasonable character classification performance with average accuracies across all scenarios of over 80%, including the highest accuracies in the case of palette swaps. These results would be expected to improve further if character templates were expanded to include palette swaps for each character.

Animation recognition proved considerably more difficult. Accuracy remained below 15% across all three stages, largely because the template database contained only a subset of the possible animation frames and many animations share similar poses. Constructing a complete database was impractical, particularly when dynamic visual effects and the large number of animation states are considered. Similar poses between different animations also made appearance-only classification ambiguous. These limitations demonstrate that recognizing a character’s identity is substantially easier than determining its exact animation state from an isolated frame.

The primary remaining challenges are full and partial character occlusion, dynamic visual effects, and changes in stage appearance. These conditions can

introduce regions that resemble characters or contaminate the shape and texture descriptors used for matching. Future improvements could include a more complete animation template database and multi-frame temporal smoothing. Incorporating temporal player-state information could also help resolve visually similar animation states that cannot be reliably distinguished from a single frame.

Overall, the project demonstrates that classical image processing techniques can provide a practical foundation for extracting gameplay information directly from Super Smash Bros. Melee video. More importantly, the experiments show that the effectiveness of the overall system depends heavily on obtaining reliable detections before applying increasingly complex tracking and matching techniques. For this particular problem, careful image segmentation and understanding of the visual characteristics of the game proved more important than simply adding additional feature descriptors. While the current system is not sufficiently robust for complete animation-level gameplay analysis, it provides a useful basis for extracting higher-level information such as active character identities, scree-space locations, HUD elements, and other gameplay metadata.

References

- [1] Bewley, Alex, et al. "Simple Online and Realtime Tracking." 2016 IEEE International Conference on Image Processing (ICIP), 1 Sept. 2016, pp. 3464-3468, <https://arxiv.org/abs/1602.00763>, 10.1109/ICIP.2016.7533003.
- [2] Kozłowski, Kamil, et al. "Visual Analysis of Computer Game Output Video Stream for Gameplay Metrics." *Lecture Notes in Computer Science*, vol. 12141, 2020, pp. 538-552, 10.1007/978-3-030-50426-7_40.
- [3] Amraji, N., Mu, L., Milanova, M. (2011). Shape-Based Human Actions Recognition in Videos. In: Jacko, J.A. (eds) Human-Computer Interaction. Design and Development Approaches. HCI 2011. *Lecture Notes in Computer Science*, vol 6761. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-21602-2_5
- [4] Zhang, Dengsheng, and Guojun Lu. "A Comparative Study of Curvature Scale Space and Fourier Descriptors for Shape-Based Image Retrieval." *Journal of Visual Communication and Image Representation*, vol. 14, no. 1, Mar. 2003, pp. 39-57, 10.1016/s1047-3203(03)00003-8.
- [5] Ming-Kuei Hu, "Visual pattern recognition by moment invariants," in *IRE Transactions on Information Theory*, vol. 8, no. 2, pp. 179-187, February 1962, doi: 10.1109/TIT.1962.1057692.